FSD Mini Project Report

CampusShield

Cyber Incident Reporting and Evidence Tracking Portal



Student name	Pratham Singh
PRN	1032211384
Course name	Full Stack Development

Abstract

CampusShield is a full stack web application that gives a college one place to report, track and investigate cyber incidents. It was built with HTML, CSS, JavaScript, React, Node.js, Express and MongoDB.

Students report incidents such as phishing or account compromise and can attach an evidence file. Admins review every case, update its status and add investigation notes. The system stores a SHA-256 hash for each evidence file, so a later check can show whether the stored file has changed. An audit log records logins and case actions.

The application passed all 24 API checks and all 12 browser checks. Testing confirmed that a changed evidence file was detected, that students could not open other students' cases, and that students could not use admin routes. The project shows that a small MERN application can support a clear incident workflow with basic evidence integrity. It was tested with sample data only, so real users and hosted deployment remain future work.

1. Introduction

1.1 Background and motivation

College students and staff depend on email, online portals and shared devices. A phishing message or a compromised account can therefore affect many people quickly. Without a single reporting channel, reports can spread across emails and phone calls.

This project was chosen because it connects full stack development with cyber security and forensics. It applies web development skills to a practical problem: keeping incident records and evidence organised and checkable.

1.2 Problem statement

A college needs one secure place where incidents can be reported, tracked and investigated. It also needs a way to show whether stored evidence has changed since it was submitted.

1.3 Objectives

1. Provide secure registration and login.
2. Give students and admins different access.
3. Let students report incidents with an optional evidence file.
4. Let admins update case status and add investigation notes.
5. Detect changes to stored evidence using SHA-256.
6. Record important actions in an audit log.

7. Give admins a dashboard of incidents by type, severity and status.
8. Provide a responsive interface for laptops and phones.

1.4 Scope

The project covers incident reporting, case tracking, evidence integrity checks, an audit log and an admin dashboard for one college. It has two roles: student and admin.

1.5 Limitations

- It runs on a local computer and is not deployed publicly.
- Evidence files are stored on disk without encryption.
- It does not send email alerts.
- The audit log is stored in the same database as the application data, so it is not independently protected.
- It was tested with sample data, not with real college users.

2. Literature Review

CampusShield combines three ideas from existing work: structured incident handling, evidence integrity, and secure web application design. Existing platforms are powerful, but they are built for security or service teams, not as a small student-facing college portal.

2.1 Incident handling guidance

NIST's Computer Security Incident Handling Guide describes incident response as an organised process of preparation, detection and analysis, containment and recovery, and post-incident activity [1]. NIST's 2025 update connects incident response to wider cybersecurity risk management under the NIST Cybersecurity Framework 2.0 [2]. CampusShield applies this idea at a small scale through recorded reports, case status and notes.

2.2 Evidence handling and integrity

NIST guidance on integrating forensic techniques into incident response stresses preserving data integrity and keeping records of how evidence is handled [3]. SHA-256 is a hash function defined in the Secure Hash Standard [4]. CampusShield uses it to compare a file with its original hash. The check shows whether the stored file changed, but it does not prove who changed it or whether the original was genuine.

2.3 Web application security

The OWASP Top 10 lists common web application risks, including broken access control and cryptographic failures [5]. CampusShield addresses access control by checking the user's role and case ownership on the server. Passwords are stored with bcrypt, a deliberately slow password

hashing method [6]. Login sessions use JSON Web Tokens, a standard signed token format [7]. The API follows the REST architectural style [8].

2.4 Existing systems

System	Purpose	Strength	Gap for this project
TheHive [9]	Security incident response platform	Built for security teams	More than a small college needs
Jira Service Management [10]	IT service and request management	Mature ticket workflows	General purpose, not focused on evidence integrity
Email	Informal reporting	Familiar and easy	No shared status, audit log or integrity check

2.5 State of the art

Current practice treats incident response as part of ongoing risk management, not as a one-off task [2]. It also expects access control and record keeping to be built into systems from the start [3][5]. CampusShield follows these principles in a simplified form suited to a student project.

3. Methodology

CampusShield follows a three-tier design: a React client, an Express server and a MongoDB database. The project was built by one student in short iterations, with each feature tested before the next was added.

3.1 Development process

An iterative approach based on Agile ideas was used:

1. Define the features and data model.
2. Build the API and test it with direct requests.
3. Build the React pages.
4. Test the complete flows in a real browser.
5. Fix issues, then retest.

3.2 Technologies and tools

Layer	Technology	Reason for choice
Structure	HTML5	Standard page structure that React renders into
Styling	CSS3	Responsive layouts with media queries
Logic	JavaScript	One language for the front end and back end
User interface	React with Vite	Reusable components, state and fast builds
Navigation	React Router	Moves between pages without a full reload
Server	Node.js and Express	Simple REST API with middleware for login checks
Database	MongoDB with Mongoose	Documents suit cases with notes and evidence details
Security	JWT, bcryptjs, SHA-256	Login tokens, password hashing and evidence integrity
File upload	Multer	Handles files with type and size limits

3.3 System architecture and process flow

Figure 1 shows how the three tiers connect, and Figure 2 shows how a case moves between the student, the system and the admin.

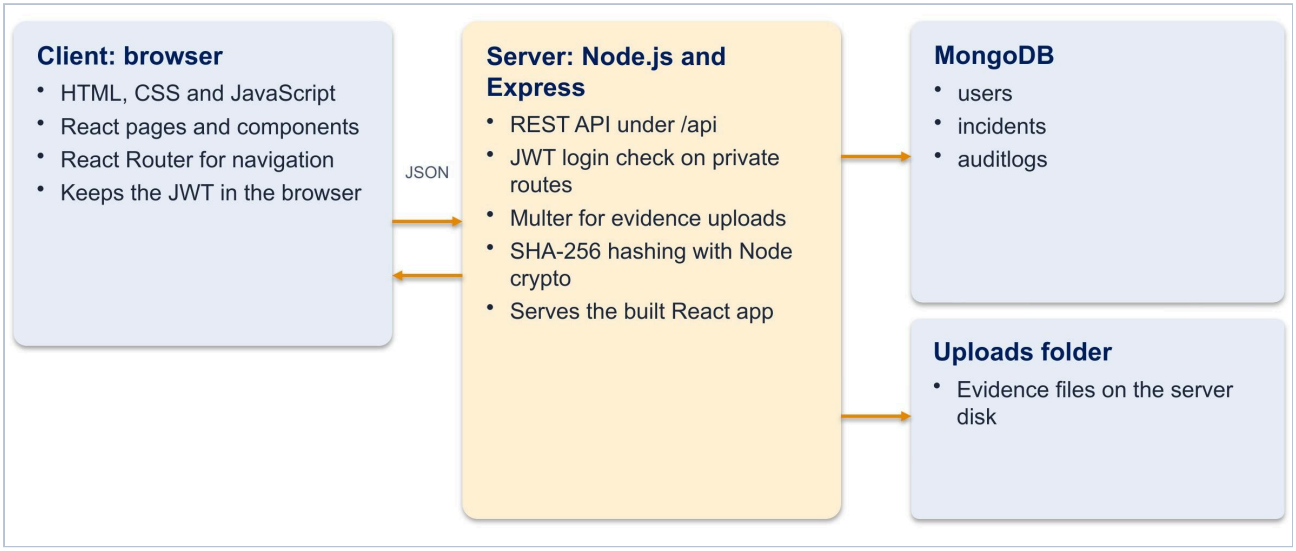


Figure 1: System architecture

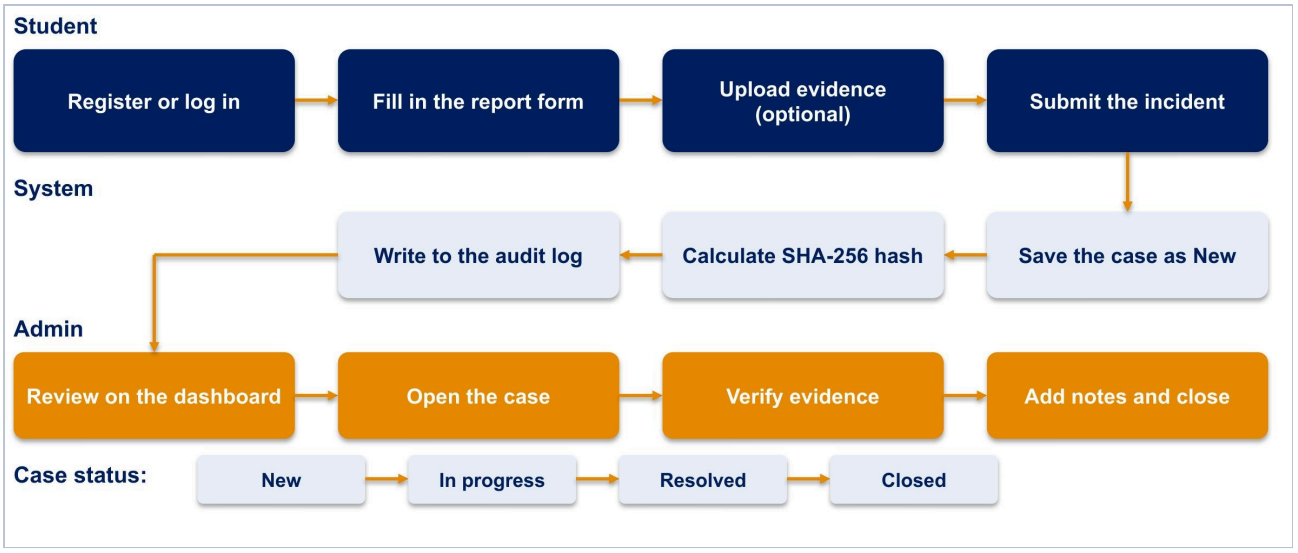


Figure 2: Process flow

3.4 Front end

The front end is a single page application. React components provide the pages for login, incidents, reporting, case details, the dashboard and the audit log. State holds form values and loaded data, and `useEffect` loads data when a page opens. A shared `api.js` file sends requests with the login token and passes server errors to the page.

3.5 Back end

The Express server exposes a REST API under `/api`. The `requireLogin` middleware verifies the JWT, and `requireAdmin` limits admin routes. Case routes also check ownership, so a student receives a "not found" response for another student's case. Figure 3 shows the login and access control workflow.

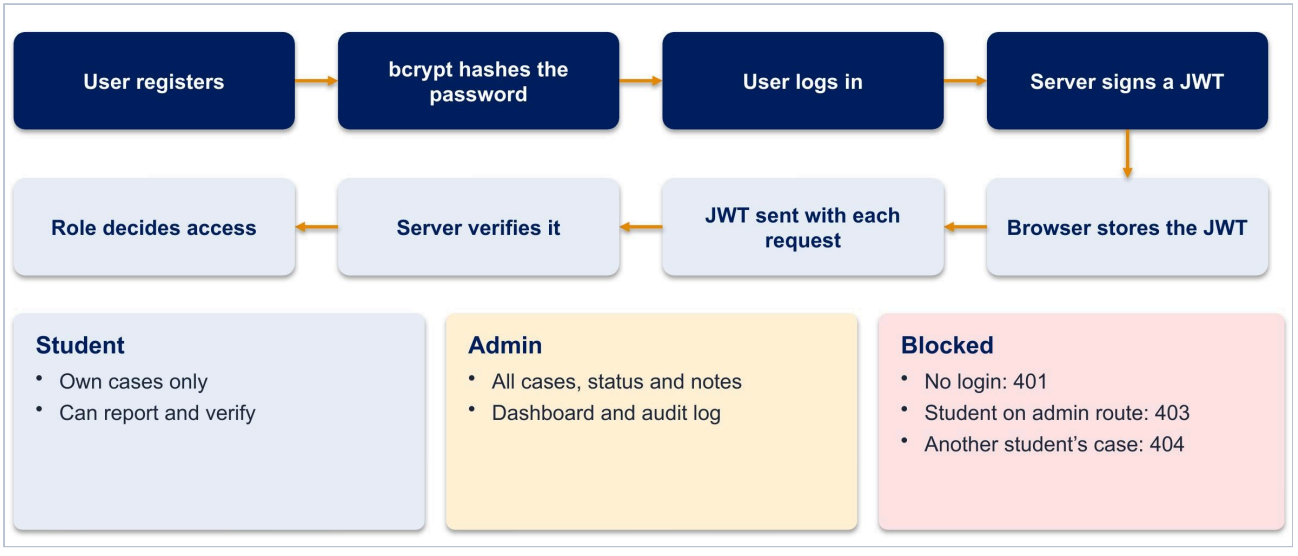


Figure 3: Login and access control workflow

3.6 Database design

Collection	Main fields	Purpose
users	name, email, passwordHash, role	Accounts and roles
incidents	title, type, severity, description, status, reporter, notes, evidence	Case records and evidence details
auditlogs	user, action, incidentTitle, timestamps	Record of actions

Notes and evidence details sit inside the incident document, so one read returns the whole case.

3.7 Evidence integrity method

When a file is uploaded, the server calculates its SHA-256 hash and stores it with the incident. When someone presses Verify integrity, the server reads the stored file, calculates the hash again and compares the two values. Matching hashes show Intact, and different hashes show Tampered. The result is written to the audit log, as shown in Figure 4.

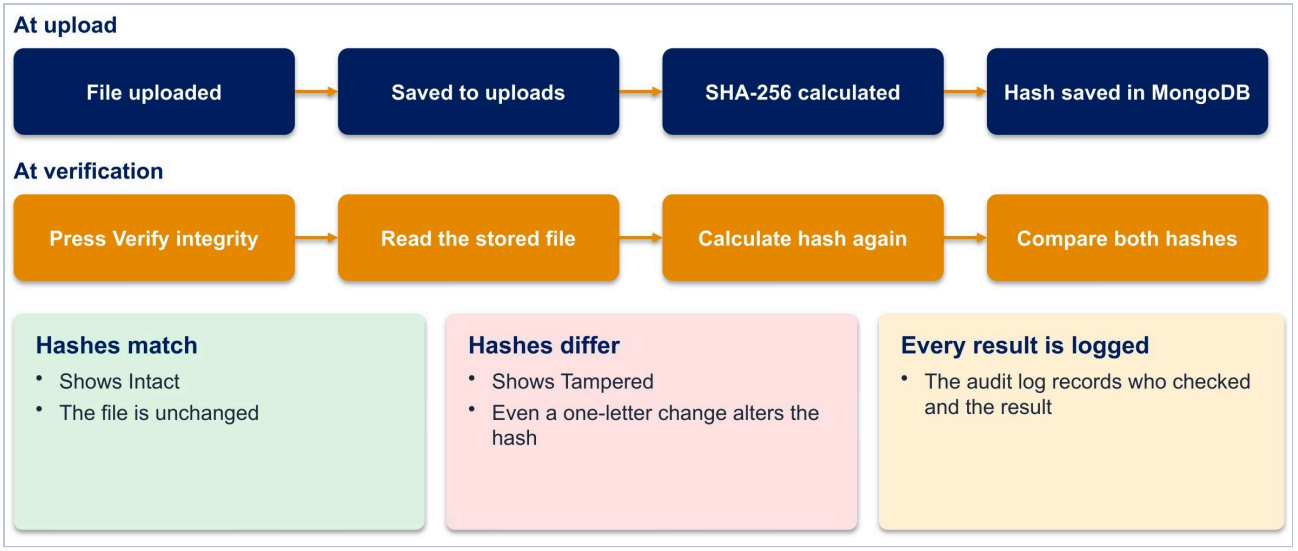


Figure 4: Evidence integrity workflow

4. Results and Discussion

The completed application met all eight objectives in testing, and no planned check failed. The tests used six sample incidents and two sample student accounts, plus one admin account.

4.1 Outcomes

The final application provides registration and login, role-based access, incident reporting, evidence upload, case status, investigation notes, integrity checks, an audit log and an admin dashboard. The interface adapts from a laptop layout to a phone layout without sideways scrolling.

4.2 Test results

Test type	Checks	Passed	Failed
Direct API checks	24	24	0
Real browser walkthrough	12	12	0

The API checks covered login errors, role changes, access control, upload limits, status updates, notes and evidence verification. The browser walkthrough used real clicks and typing as a student and as an admin.

4.3 Key findings

- **Tamper detection worked.** After a stored file was changed, Verify integrity reported Tampered. After the original was restored, it reported Intact.
- **Access control worked.** A student could not view, verify or download another student's evidence.
- **Admin routes were protected.** Students received an error for admin status, notes, statistics and audit routes.
- **Role injection failed.** Sending role as admin during registration still created a student account.
- **Upload limits worked.** An `.exe` file and a 6 MB file were rejected.



Figure 5: Integrity check before a stored file was changed



Figure 6: The same check after the stored file was changed

4.4 Challenges

Challenge	Solution
Proving evidence was not changed	Save a SHA-256 hash at upload and recalculate it during each check
Keeping cases private	Check the case owner on the server
Stopping users becoming admin	Ignore any role sent during registration
Accepting uploads safely	Allow set file types up to 5 MB and use random stored names
Avoiding leftover files	Delete the uploaded file if saving the incident fails

4.5 Evaluation against objectives

Objective	Result
Secure registration and login	Met
Different student and admin access	Met
Incident reporting with evidence	Met
Status updates and notes	Met
Change detection for evidence	Met
Audit log	Met
Admin dashboard	Met
Responsive interface	Met at laptop and phone widths

These results show that the features work as designed under test. They do not measure real user satisfaction or performance under heavy use, because no user study or load test was carried out.

5. Conclusion

CampusShield shows that a small MERN application can give a college one organised place to report and investigate cyber incidents. Its main contribution is combining a simple case workflow with evidence integrity checks and an audit log.

All planned features were built, and all 36 checks passed. Role-based access protected private cases, and the SHA-256 check detected a changed evidence file.

5.1 Recommendations for future work

- Deploy online with HTTPS and a hosted database such as MongoDB Atlas.
- Add analyst and department roles.
- Encrypt evidence files in storage.

- Send email alerts when a case changes.
- Make the audit log tamper-evident.
- Add sign-in with college accounts.
- Test with real users and measure performance under load.

6. References

References use a numbered IEEE style.

1. P. Cichonski, T. Millar, T. Grance and K. Scarfone, “Computer Security Incident Handling Guide,” NIST SP 800-61 Rev. 2, 2012. https://doi.org/10.6028/NIST.SP.800-61r2
2. NIST, “Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile,” NIST SP 800-61 Rev. 3, Apr. 2025. https://csrc.nist.gov/pubs/sp/800/61/r3/final
3. K. Kent, S. Chevalier, T. Grance and H. Dang, “Guide to Integrating Forensic Techniques into Incident Response,” NIST SP 800-86, 2006. https://doi.org/10.6028/NIST.SP.800-86
4. NIST, “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015. https://doi.org/10.6028/NIST.FIPS.180-4
5. OWASP Foundation, “OWASP Top 10.” https://owasp.org/Top10/
6. N. Provos and D. Mazieres, “A Future-Adaptable Password Scheme,” in Proc. USENIX Annual Technical Conference, 1999. https://www.usenix.org/legacy/event/usenix99/provos/provos.pdf
7. M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, May 2015. https://www.rfc-editor.org/rfc/rfc7519
8. R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm
9. TheHive Project, “TheHive.” https://thehive-project.org/
10. Atlassian, “Jira Service Management.” https://www.atlassian.com/software/jira/service-management
11. MongoDB, “MongoDB Documentation.” https://www.mongodb.com/docs/manual/
12. Meta Open Source, “React.” https://react.dev/
13. OpenJS Foundation, “Express.” https://expressjs.com/

7. Appendices

Appendix A: API endpoints

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Create a student account	Public
POST	/api/auth/login	Log in	Public
GET	/api/incidents	List incidents	Logged in
POST	/api/incidents	Report an incident	Logged in
GET	/api/incidents/:id	View a case	Owner, admin
GET	/api/incidents/:id/verify	Check evidence	Owner, admin
GET	/api/incidents/:id/evidence	Download evidence	Owner, admin
PUT	/api/incidents/:id/status	Change status	Admin
POST	/api/incidents/:id/notes	Add a note	Admin
GET	/api/admin/stats	Dashboard numbers	Admin
GET	/api/admin/audit	Audit log	Admin

Appendix B: User manual

1. Start MongoDB.
2. In the project folder, run `npm install`, `npm run seed`, `npm run build` and `npm start`.
3. Open http://localhost:4100 in a browser.
4. Students create an account, then choose Report to submit an incident.
5. Admins log in, open a case from Incidents, update its status and add notes.
6. Press Verify integrity on a case to check its evidence file.

Appendix C: Screenshots

All screenshots are from the running application. The names and incidents are sample data.

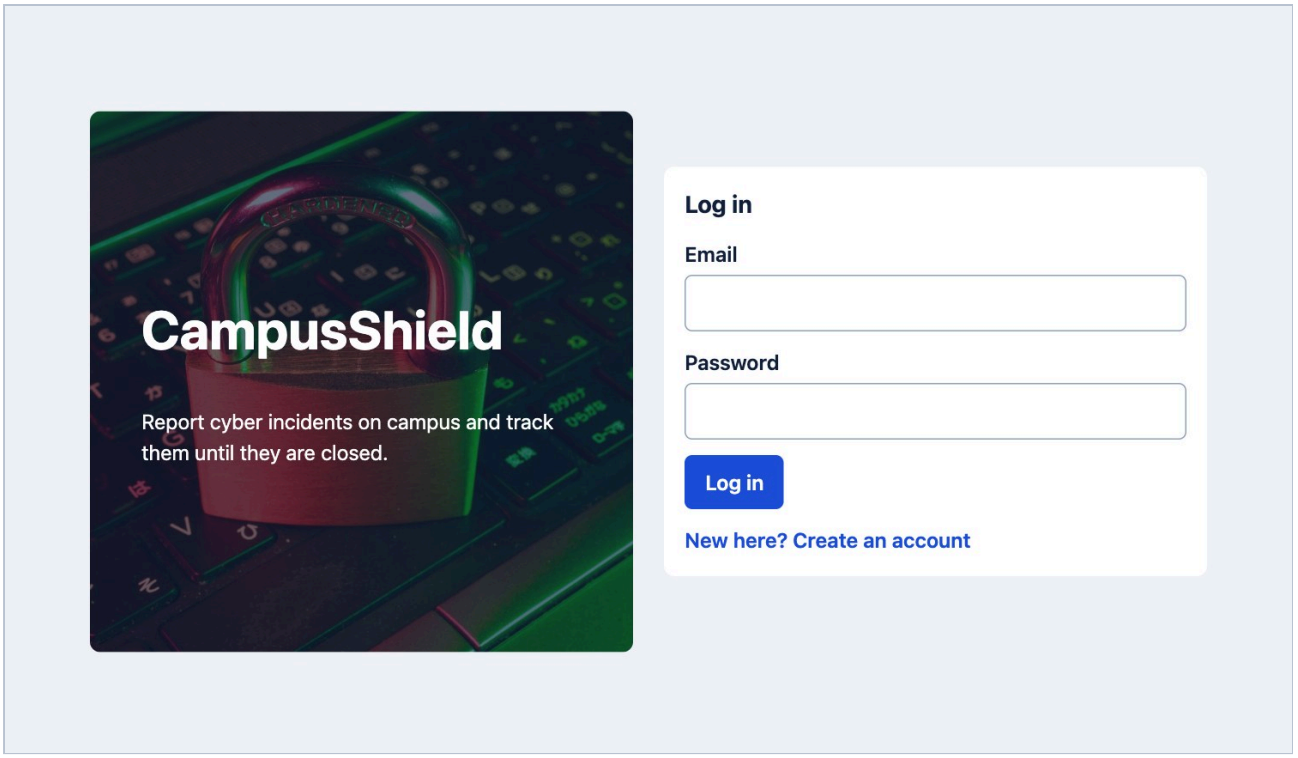


Figure 7: Login page

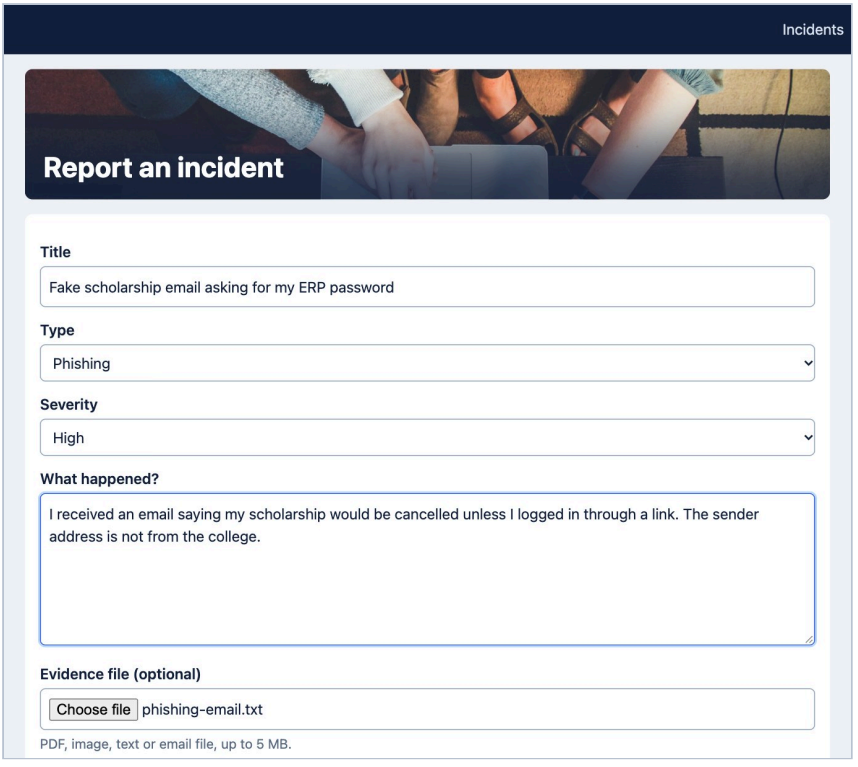


Figure 8: Report form with evidence upload

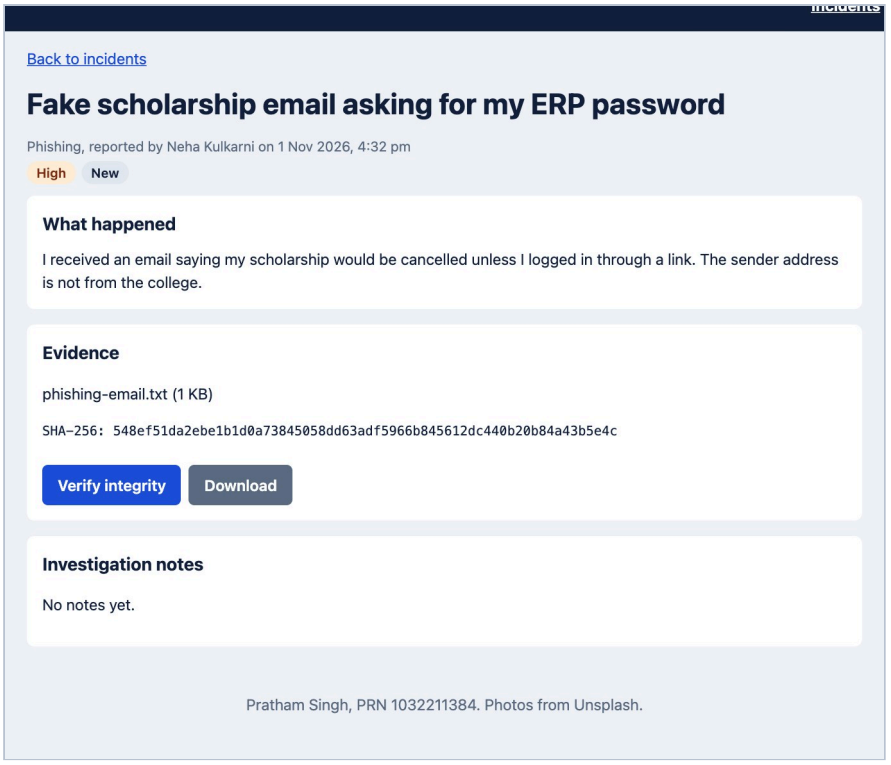


Figure 9: Case with its SHA-256 hash

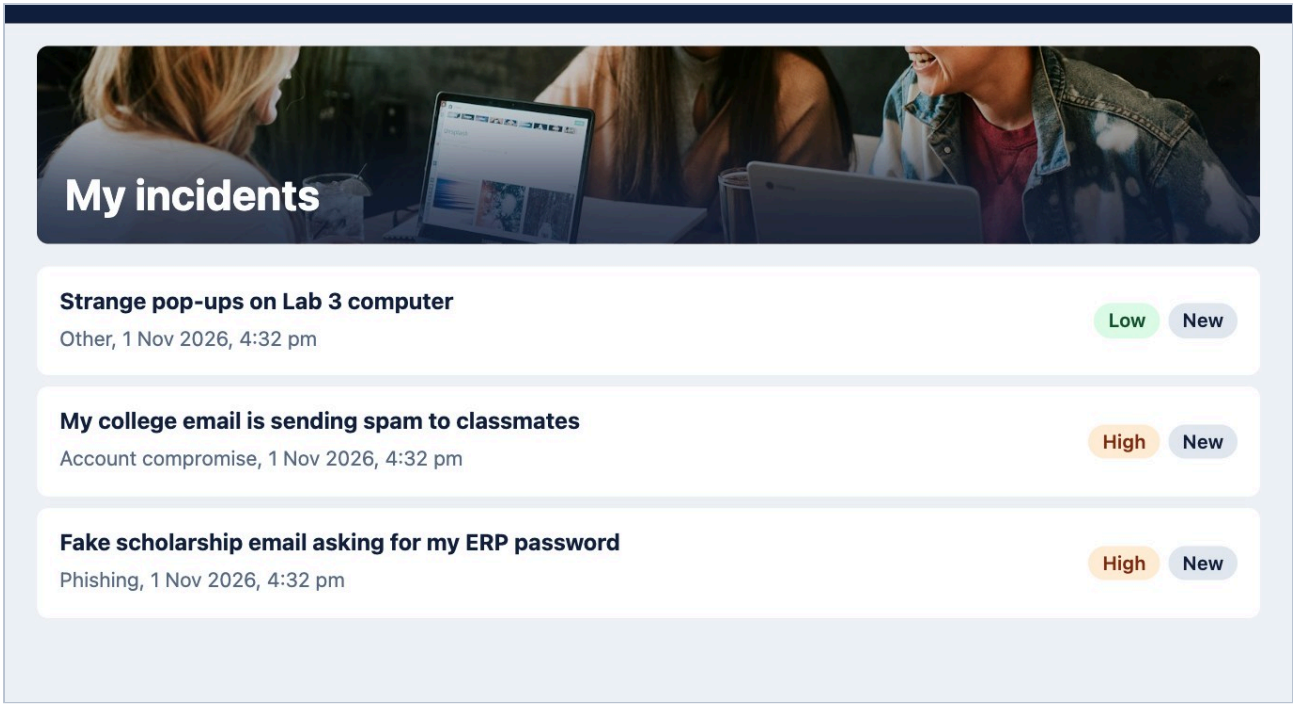


Figure 10: A student sees only their own cases

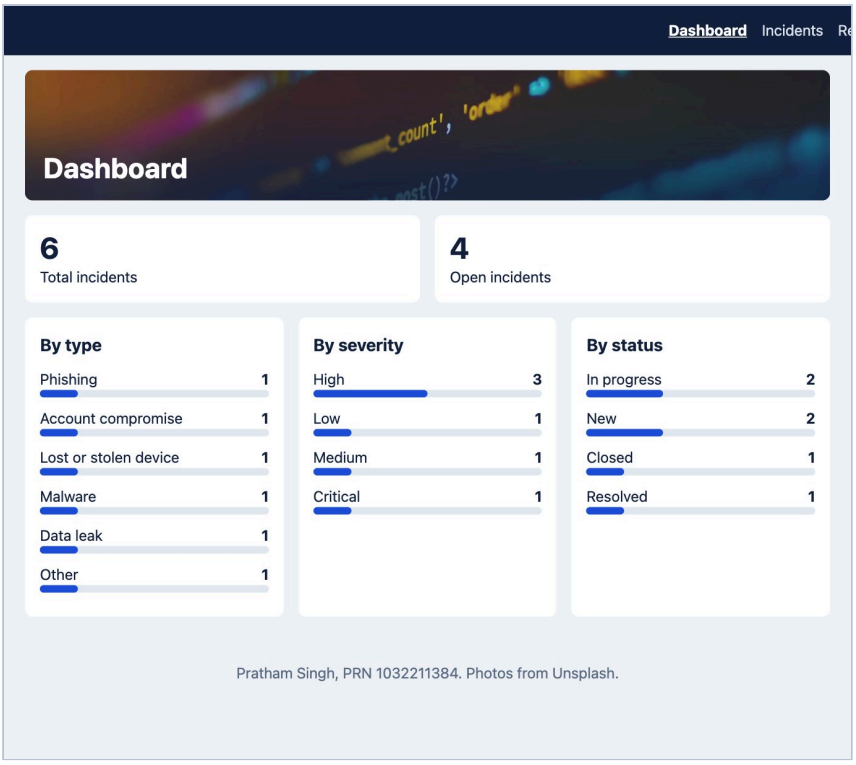


Figure 11: Admin dashboard

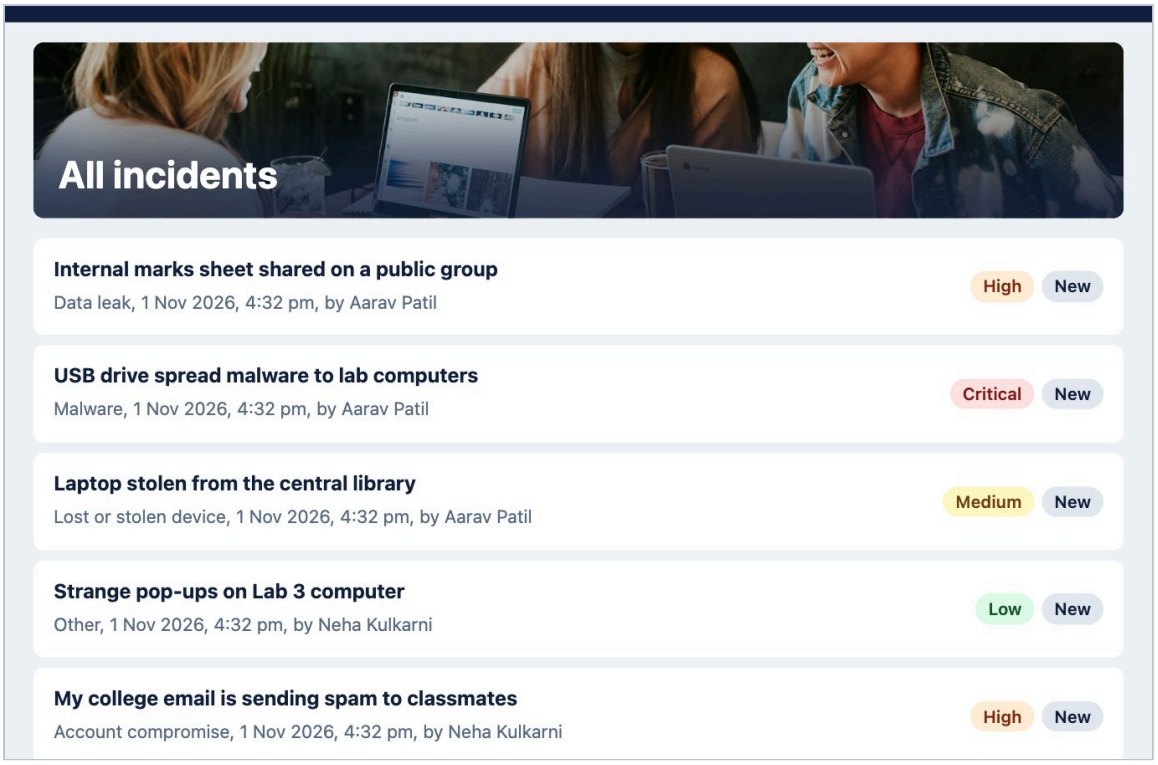


Figure 12: All incidents

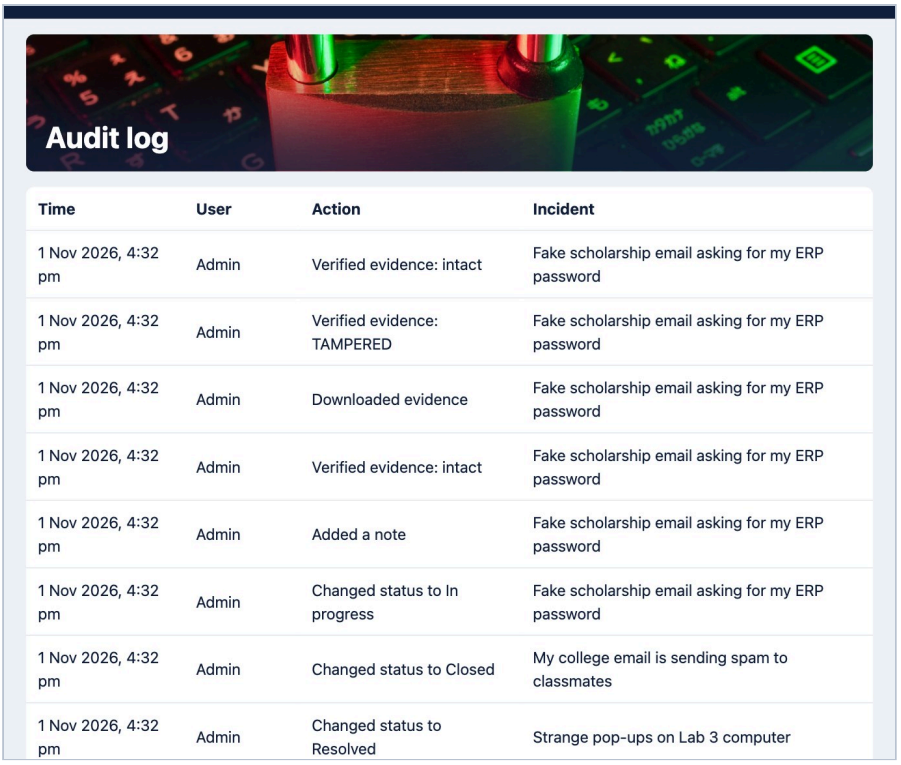


Figure 13: Audit log with the tamper result

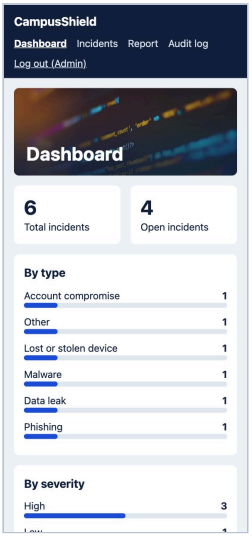


Figure 14: Dashboard at phone width

Appendix D: Source code

The source code is available on GitHub: github.com/singhpratham19/campusshield

Photos used in the application are from Unsplash, by FlyD, Brooke Cagle, John and Shahadat Rahman.